

Chapter 5: Multimedia Assets

Preparation For AI: From Raw Data To Reliable Models

Alexander D'hoore

Multimedia Assets

Not every AI dataset is a neat table.

Chapter 5 asks:

How do we train from large assets without losing dataset meaning?

The answer is object storage plus manifests.



Assets Are Not Rows

Multimedia datasets start as files:

- images
- audio recordings
- videos
- documents
- scans
- text assets

The file is the source signal, but the file does not explain itself.



The Meaning Is Small But Critical

The asset bytes are large.

The meaning is usually small:

- label
- split
- checksum
- source metadata
- grouping key
- preprocessing history

Losing that meaning breaks the dataset.



Do Not Put Everything In SQL

Large binary assets are awkward table cells.

They often need:

- specialized decoders
- batch-by-batch loading
- local caching
- GPU-friendly transforms
- separate privacy handling

The metadata is much smaller than the asset.



The Core Pattern

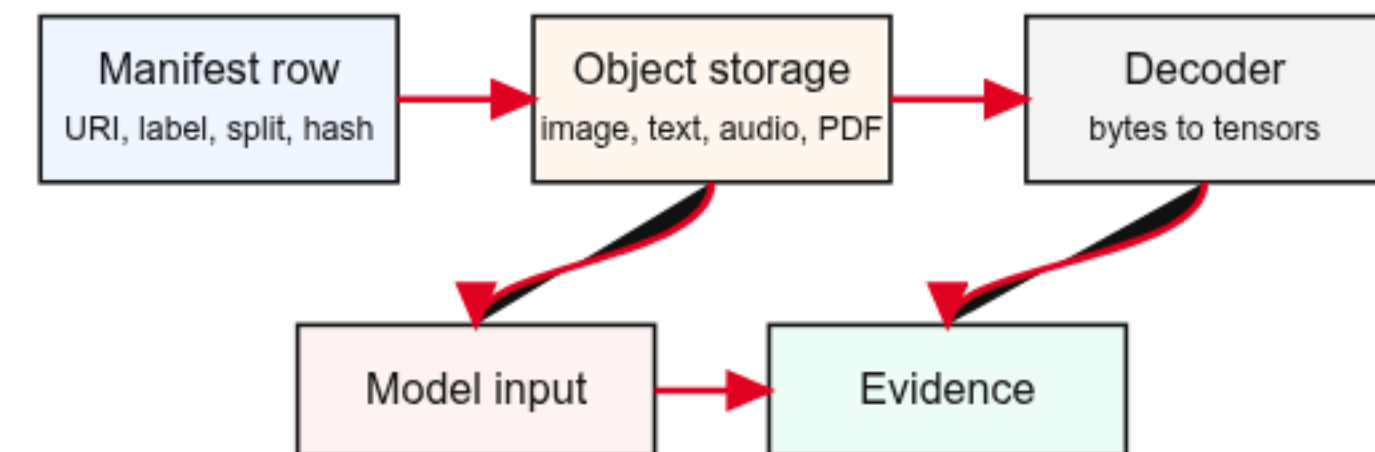
Large bytes live in object storage.

Small structured meaning lives in a manifest.

Training code reads **both**.

`asset bytes + manifest row = training example`

The manifest gives assets meaning



Folder Layouts Are Tempting

Beginner image datasets often look like:

```
train/pizza/image001.jpg  
train/ramen/image002.jpg  
test/pizza/image003.jpg
```

Many libraries can read this.

But the folder name is now carrying the label and split.

Manifest Beats Guessing

A manifest makes meaning explicit:

```
asset_uri, label, split, sha256  
s3:///.../0001.jpg, pizza, train, ...  
s3:///.../0002.jpg, ramen, train, ...
```

Now the dataset definition is queryable, versionable, and reviewable.

Folders are convenient; manifests are explicit

Folder-only meaning

```
train/pizza/img001.jpg  
train/ramen/img002.jpg  
test/pizza/img003.jpg
```

Move a file, change the meaning.

Manifest meaning

asset_uri	label	split
s3:///.../img001.jpg	pizza	train
s3:///.../img002.jpg	ramen	train

Meaning is queryable
and auditable.

What Goes In A Manifest?

Common fields:

- `asset_uri`
- `label_name`
- `split`
- `sha256`
- `content_type`
- `source dataset`
- `modality-specific metadata`

The exact columns change. The idea stays the same.

Checksums, Fingerprints, And Splits

A checksum asks:

are these exactly the bytes we recorded?

Fingerprints are quick checks:

- image size
- audio duration
- sample rate
- text length
- language

They make unexpected changes visible early.

Train, validation, and test splits are data preparation.



Leakage Is Easy

Multimedia leakage can happen when related files cross splits:

- frames from one video
- images of one physical sample
- chunks from one document
- recordings from one speaker
- augmented copies of one image

Keep related groups together.

Split real-world groups, not just files

Leaky split

same plant leaf
frame 001 -> train
frame 002 -> test

Model sees the same object twice.

Group-aware split

sample_id = leaf_42
all related files -> train
or all related files -> test

Evaluation becomes more honest.

Version The Relationship

For asset datasets, this is the durable pattern:

```
keep heavy assets stable  
version the manifest  
record derived assets and preprocessing
```

The dataset version is the relationship between files, labels, splits, and metadata.



Caches Are Not Datasets

A local cache is allowed to disappear.

It exists to make training faster.

The dataset should still be reproducible from:

- object storage
- the manifest
- the code



Training From A Manifest

In PyTorch, the shape is:

```
manifest -> Dataset -> DataLoader -> model loop
```

The Dataset turns one manifest row into one example.

The DataLoader batches examples for training.



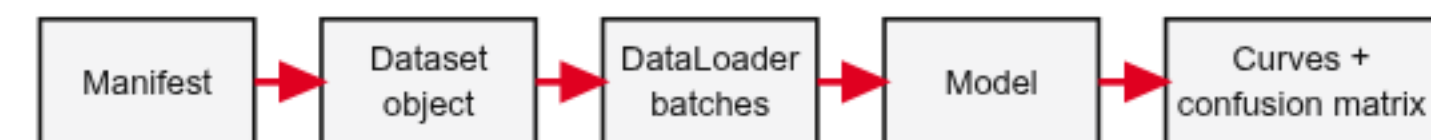
Input Pipeline Matters

A GPU can only train while it has data.

The pipeline may bottleneck on:

- object storage throughput
- image decoding
- text tokenization
- transforms
- batch size
- worker processes

A neural network still starts with data plumbing



If the input pipeline stalls, the GPU waits.

Modality Changes The Decoder

The platform pattern stays the same.

The decoder changes:

- images: load pixels, resize, augment, batch tensors
- text: tokenize, pad, build attention masks
- audio: decode waveform, sample rate, duration

Different asset type. Same manifest discipline.



Lab Orientation

You train from two manifest-backed datasets:

- Food-101 images
- Banking77 texts

Both follow:

`manifest -> asset bytes -> decoder -> tensors -> model -> evidence`

The asset type changes. The data contract does not.



Pretrained Models Make This Practical

The labs use pretrained models:

- MobileNet for images
- DistilBERT for text

The model already knows general visual or language patterns.

The lab fine-tunes those representations for a small selected task.



What To Notice In The Lab

Look for:

- selected labels
- manifest-defined splits
- decoded batches
- training curve
- confusion matrix
- prediction examples
- mistakes, not only accuracy

Evidence matters more than a single score.

Multimedia Assets

Assets live in object storage.

Manifests define **meaning**.

Splits and checksums are part of the dataset.

Training reads assets in batches.

Outputs are evidence that the dataset and model behavior are understandable.

Next: automate repeatable work with orchestration.